

Course 50.050/50.550

Advanced Algorithms

Week 6 – Cohort Class L06.03



Outline of Cohort Class

- ▶ Recap: Graphs and graph algorithms
- ▶ **Mini-quiz 2**
- ▶ Class activity: Laying TV cables the prim and proper way
- ▶ Kruskal's algorithm
- ▶ Youtube video: Splitting the rent of an apartment



Recap: Graphs and basic structure

Definition: An **graph** is a 3-tuple $G = (V, E, \phi)$, where V is the **vertex set**, E is the **edge set**, and ϕ is the **incidence function**.

Case: G is undirected.

- ▶ $\deg_G(x)$ = number of edges in E incident to x .
- ▶ $2|E| = \sum_{v \in V} \deg(v)$.
- ▶ G has even number of odd-degree vertices.

Case: G is directed.

- ▶ $\deg_G^-(x)$ (**in-degree**) = number of edges with x as the head.
- ▶ $\deg_G^+(x)$ (**out-degree**) = number of edges with x as the tail.
- ▶ $|E| = \sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v)$.

Definition: Let $G = (V, E, \phi)$ be a graph.

- ▶ The **vertex-induced subgraph** of G **induced by** $U \subseteq V$ is the unique subgraph $G[U]$ with vertex set U with the **largest possible edge set**.
- ▶ The **edge-induced subgraph** of G **induced by** $D \subseteq E$ is the unique subgraph $G[D]$ with edge set D with the **smallest possible vertex set**.

Definition: Let $G = (V, E, \phi)$ be an undirected graph with no loops.

- ▶ A **vertex coloring** is an assignment of colors to the vertices, such that **adjacent vertices** are assigned **different colors**.
 - ▶ The **chromatic number** of G = smallest number of colors needed for a vertex coloring of G .
- ▶ An **edge coloring** is an assignment of colors to the edges, such that any two edges incident to a common vertex are assigned **different colors**.
 - ▶ The **edge chromatic number** of G = smallest number of colors needed for an edge coloring of G .



Recap: Graph isomorphism

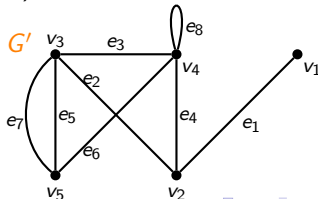
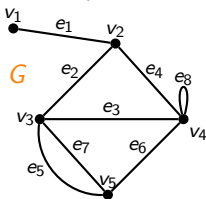
Definition: Let G, G' be **undirected** graphs. We say that G and G' are **isomorphic** graphs if there exist two **bijections** $f : V(G) \rightarrow V(G')$ and $g : E(G) \rightarrow E(G')$ such that for all $v \in V(G)$, $e \in E(G)$:

- ▶ Vertex v is incident to the edge e if and only if vertex $f(v) \in V(G')$ is incident to the edge $g(e) \in E(G')$.

Definition: Let G, G' be **directed** graphs. We say that G and G' are **isomorphic** graphs if there exist two **bijections** $f : V(G) \rightarrow V(G')$ and $g : E(G) \rightarrow E(G')$, such that for all $u, v \in V(G)$, $e \in E(G)$:

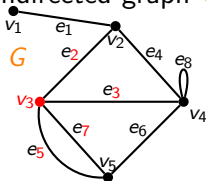
- ▶ Vertices u, v are the head and tail, respectively, of the edge e if and only if vertices $f(u), f(v)$ are the head and tail, respectively, of the edge $g(e)$.

$G = (V, E, \phi)$ and $G' = (V', E', \phi')$, as depicted below, are isomorphic.



Recap: Incidence matrices

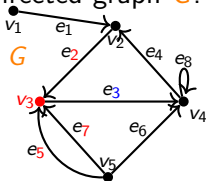
Example: Undirected graph G . Incidence matrix M of G :



$$M = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

- ▶ Column sum of j -th column of $M = \begin{cases} 2, & \text{if } e_j \text{ is a non-loop;} \\ 1, & \text{if } e_j \text{ is a loop.} \end{cases}$
- ▶ If G has no loops, then row sum of i -th row of M equals $\deg(v_i)$.

Example: Directed graph G .



Incidence matrix M of G :

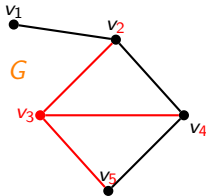
$$M = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & -1 & -1 & 0 \end{bmatrix}$$

- ▶ The column sum of every column of M always equals 0.
 - ▶ e_j is a loop if and only if the j -th column of M is the zero vector.
- ▶ $\deg^-(v_i) = (\text{number of } 1\text{'s in } i\text{-th row}) + (\text{number of loops at } v_i)$.
- ▶ $\deg^+(v_i) = (\text{number of } -1\text{'s in } i\text{-th row}) + (\text{number of loops at } v_i)$.



Recap: Adjacency matrices (for simple graphs)

Undirected case:

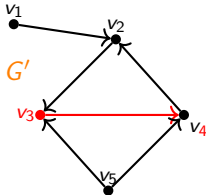


Adjacency matrix A of G :

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- ▶ A is symmetric in the undirected case.
- ▶ The row sum of the i -th row equals the column sum of the i -th column.
 - ▶ This common row/column sum equals the **degree** $\deg(v_i)$.

Directed case:



Adjacency matrix A' of G' :

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- ▶ The row sum of the i -th row equals the **out-degree** $\deg^+(v_i)$.
- ▶ The column sum of the j -th column equals the **in-degree** $\deg^-(v_j)$.



Recap: Connectedness and connected components

Definition: Let $G = (V, E, \phi)$ be an **undirected** graph. If u and v are connected for all possible vertices u, v in V , then we say that G is **connected**, otherwise we say that G is **disconnected**.

- ▶ A **connected component** of G is a connected vertex-induced subgraph $G[U]$ of G for some subset $U \subseteq V$, such that $G[U']$ is disconnected for all $U \subsetneq U' \subseteq V$ (if any).
- ▶ **DFS** can be used to compute connected components of G .

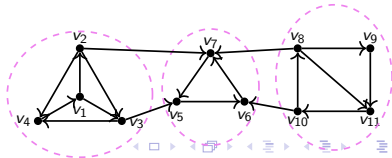
Definition: Let $G = (V, E, \phi)$ be a **directed** graph.

- ▶ We say that G is **weakly connected** if its underlying undirected graph is connected.
- ▶ We say that G is **strongly connected** if for all vertices u, v in V , there is a walk from u to v , and there is a walk from v to u .
- ▶ **Kosaraju's algorithm** can be used to compute SCCs in G .

Example: An undirected graph with 4 connected components.



Example: A weakly connected directed graph with 3 SCCs.



Recap: Special graphs

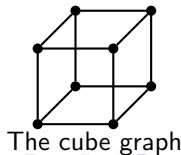
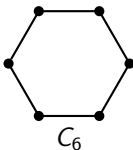
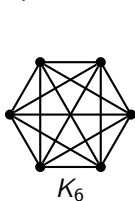
Definition: A graph G is **bipartite** if there is a **bipartition** (V_1, V_2) of $V(G)$ such that every edge has one endpoint in V_1 , one endpoint in V_2 .

- ▶ Let G be an undirected graph with no loops. Then “ G is bipartite” $\Leftrightarrow$ “ G is 2-colorable” $\Leftrightarrow$ “ G has no odd cycle as a subgraph”.

Definition: A graph G is **planar** if there exists a plane drawing of G .

- ▶ A **plane graph** is a pair (G, Ψ) , where G is a planar graph, and Ψ is a plane drawing of G . The dual graph of (G, Ψ) is planar.
- ▶ **Euler's formula:** If (G, Ψ) is a connected undirected plane graph with n vertices, m edges, and f faces, then $n - m + f = 2$.

Examples: Complete graph, complete bipartite graph, cycle graph, path graph, cube graph.



Mini-quiz 2 (15 mins)

Only writing materials are allowed. No calculators, notes, books, or cheatsheets are allowed. Don't worry, you won't need calculators.

If you are not present in class at the start of the quiz, you will not be given additional time to finish the quiz.

Remarks:

- ▶ There are no make-up mini-quizzes! If you arrive in class after the mini-quiz ends, or do not attend that cohort class, you will not have a chance to take the mini-quiz.
- ▶ To take into account unforeseen circumstances (e.g. mini-quiz missed due to illness), only the **best 3 of 4** mini-quiz scores will be counted towards your final grade.



After our successful bird conservation efforts,
and after our exciting cake-cutting event,
we have now started a new TV show!



It's called "**Birds and Cakes**"!
Many of my friends have bought new TVs to
watch the show, but there is a problem..



TV cables have not yet been installed in the neighborhood! Fortunately, **Red Lobster** has volunteered to lay down the TV cables.



Laying down TV cables the prim and proper way

In John's world, everyone is superstitious:

- ▶ For any two friends A and B , they do not want a “direct” TV cable joining their homes unless either they are directly linked on TweetTweet.
- ▶ We say A and B are **directly linked** on TweetTweet if A is following B on TweetTweet, or B is following A on TweetTweet (or both).
- ▶ On TweetTweet, every TweetTweet user is aware of all TweetTweet users who are directly linked to him/her/it on TweetTweet.

Red Lobster cares about the bird environment and does not want to waste more TV cable than necessary.

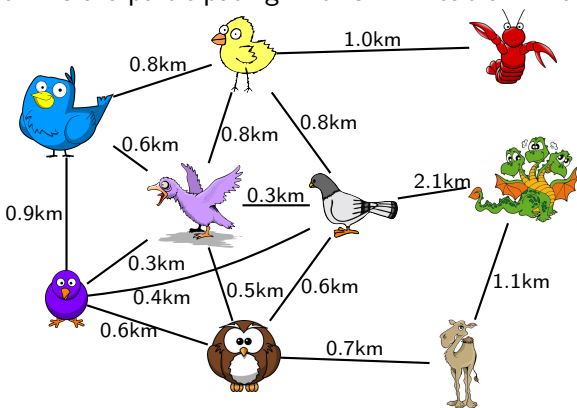
- ▶ **Task:** Can you help 🦞 to figure out the minimum total length of TV cables required to link up everyone by TV cables?



Class Activity: Laying TV cables (15 mins)

Here are the possible routes for laying TV cables.

- ▶ Route distances between homes are indicated. Only those who bought TVs are participating in this “TV cable link-up”.



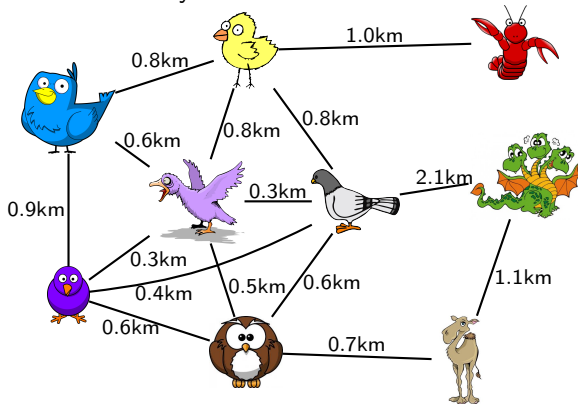
Questions:

- ▶ Find a configuration for the TV cables so that the total TV cable length is minimized. What is this minimized length?
- ▶ **Hint:** There is a **prim** and proper way to do this.



Laying TV cables: A bird's eye view

Like you, 🐦 has a bird's eye view of all route distances.



Questions:

- ▶ Is there another way to compute a minimum spanning tree not using Prim's algorithm?
 - ▶ Is there a way to find a TV cable configuration of minimized length, without starting from 🦀 and growing 🦀's network of TV cables?

Kruskal's algorithm: Initial informal pseudocode

function KRUSKALINITIAL(G, w)

Require: $G = (G.V, G.E, G.phi)$ is a connected undirected graph.

Require: w represents a weight function for G , where $w[e]$ is the weight of e .

- 1: Sort the edges in $G.E$ in ascending order according to weight
- 2: Initialize new graph F with vertex set $G.V$ and with empty edge set
- 3: **for** each edge e in $G.E$ (in sorted order) **do**
- 4: $\{u, v\} \leftarrow \phi(e)$
- 5: **if** vertices u and v are in distinct connected components of F **then**
- 6: Append array $F.E$ by e
- 7: $F.phi[lightEdge] \leftarrow G.phi[lightEdge]$
- 8: **return** F

Some Remarks:

- ▶ In Line 1, we could use any sorting algorithm of our choice.
 - ▶ e.g. if we use mergesort, then the time complexity for executing Line 1 is $O(|G.E| \log(|G.E|))$.
- ▶ **Question:** In Line 5, how do we check whether u and v are in distinct connected components?
 - ▶ We could use DFS to compute the connected components of F in each iteration, then check if u and v are in different connected components. Can we do better?



Brief introduction to disjoint-set data structure

Recall: A **data structure** is a format to **store and organize** data, in order to facilitate access and modification.

- ▶ Some data structures we know: arrays, linked lists, heaps, BSTs, hash tables.

Disjoint-set (also called **union-find**) is a data structure.

- ▶ **Main purpose:** To store a set $\{A_1, \dots, A_k\}$ of disjoint non-empty sets, so that merging and comparing disjoint sets are efficient.
 - ▶ **Key Idea:** Each A_t is identified by a representative element of A_t . Every element in A_t is assigned to this same representative.
 - ▶ Hence, two elements a, a' are in the same subset A_t if and only if their corresponding representatives are identical.

Given elements x, y , this data structure has the following operations:

- ▶ **MAKE-SET**(x): Create a new singleton set $\{x\}$ with x as the representative of this set.
- ▶ **UNION**(x, y): Merge the set containing x with the set containing y , so that every element of the merged set is assigned to the same representative.
- ▶ **FIND-SET**(x): Returns the representative of the set that x is in.

Only if you are interested: Implementation details are found in [CLRS] Chap. 21.



Improved pseudocode for Kruskal's algorithm

function KRUSKAL(G, w)

Require: $G = (G.V, G.E, G.phi)$ is a connected undirected graph.

Require: w represents a weight function for G , where $w[e]$ is the weight of e .

```
1: Sort the edges in  $G.E$  in ascending order according to weight
2: Initialize new graph  $F$  with vertex set  $G.V$  and empty edge set
3: for each vertex  $v \in G.V$  do
4:   MAKE-SET( $v$ )
5: for each edge  $e$  in  $G.E$  (in sorted order) do
6:    $\{u, v\} \leftarrow G.phi(e)$ 
7:   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then
8:     Append array  $F.E$  with element  $e$ 
9:      $F.phi[e] \leftarrow G.phi[e]$ 
10:    UNION( $u, v$ )
11: return  $F$ 
```

Remark:

- If we use the most efficient implementation of the disjoint-set data structure (so that MAKE-SET, FIND-SET and UNION are fast), then the time complexity of KRUSKAL(G, w) is $O(|G.E| \log(|G.E|))$.

Three of our friends did not buy TVs because they did not have a fixed home. So they have decided to rent an apartment together.



Renting an apartment

Three friends have found an apartment to rent.



- ▶ The apartment has 3 rooms of different sizes.
- ▶ The monthly total rent for the entire apartment is \$1000, which they have to decide how to split.
 - ▶ Let's denote the rent (in dollars) for the three rooms be α, β, γ , where $\alpha + \beta + \gamma = 1000$.
 - ▶ Assume for simplicity that no matter how the total rent is split, α, β, γ must be integers, i.e. \$1 is the smallest denominator.
- ▶ **Big headache:** Our three friends have their own different views on what a “best room” is, and how much they are willing to pay.
 - ▶ e.g. bigger is “better”, closer to main door is “better”, as much exposure to morning sun is “better”, etc.
 - ▶ For any given rent split, it is possible that our 3 friends have different opinions on which is the “best” room to pick.



Splitting the rent (15 mins)



Discussion: Is it possible to systematically find some 3-tuple (α, β, γ) for the splitting of the \$1000 rent, and assign **different** rooms to them so that all of them unanimously agree the following:

- ▶ For each of them, if there is any other room that is (in their own opinion) strictly better than their assigned room, then it is possible to increase the rent of one of the 3 rooms by \$1, and decrease the rent of one of the remaining 2 rooms by \$1, so that they would prefer their assigned room.
 - ▶ Each of them has their own individual “utility function”, assigning their own perceived values to the 3 rooms for any given split of rent.

Assumption:

- ▶ All of them prefer not paying rent over paying rent.

Youtube Video: Mathologer’s video on the rental harmony problem

- ▶ <https://www.youtube.com/watch?v=7s-YM-kcKME>



Summary

- ▶ Recap: Graphs and graph algorithms
- ▶ **Mini-quiz 2**
- ▶ Class activity: Laying TV cables the prim and proper way
- ▶ Kruskal's algorithm
- ▶ Youtube video: Splitting the rent of an apartment